

Ejercicio Evaluable 2: Sockets TCP

Sistemas Distribuidos 2025/26

Alicia Mei García Morín → 100498828
Ana Grima Vázquez de Prada → 100495785

1. Diseño de la arquitectura del sistema

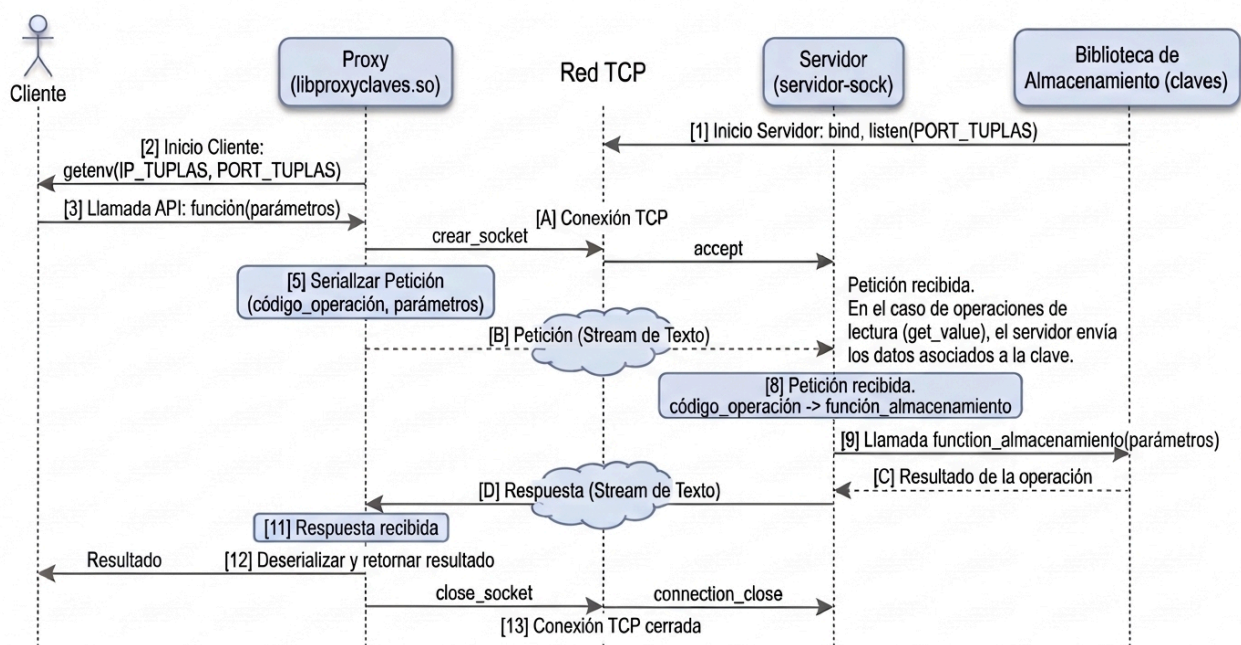
1.1. Componentes

En esta práctica hemos desarrollado un sistema con arquitectura cliente-servidor distribuida que se basa en sockets TCP. Se compone de los siguientes elementos:

- **Cliente** ([app-cliente.c](#)), que se encarga de ejecutar la batería de pruebas y de invocar las funciones de la API del servicio (*set_value*, *get_value*, *modify_value*, *delete_key*, *exist* y *destroy*).
- **Proxy** ([proxy-sock.c](#)), que es el intermediario entre el cliente y el servidor. No solo implementa las funciones de la API del servicio y transforma cada llamada local en los mensajes que se envían al servidor mediante sockets, sino que también recibe las respuestas del servidor.
- **Servidor** ([servidor-sock.c](#)) usa la biblioteca [libclaves.so](#) para almacenar las claves y atender las peticiones (múltiples de forma concurrente). Es decir, escucha las conexiones que entran en cierto puerto TCP especificado en tiempo de ejecución y crea un hilo independiente para cada cliente que se conecta.
- **Biblioteca de almacenamiento** ([claves.c](#)) es la que contiene la implementación del servicio de almacenamiento de claves en memoria.

1.2. Funcionamiento general

A continuación se muestra el flujo de comunicación cliente-servidor, donde los números representan el orden secuencial de las acciones y las flechas indican el intercambio de mensajes entre los distintos elementos del sistema:



2. Diseño del servidor (*servidor-sock.c*)

2.1. Gestión de conexiones TCP

El servidor se implementa utilizando sockets TCP, como hemos visto en clase. Al comenzar su ejecución, el servidor recibe como argumento el número de puerto en el que debe escuchar peticiones entrantes. A continuación, crea un socket TCP y lo configura para aceptar conexiones desde cualquier dirección de red disponible. Para ello se utilizan las funciones de la API de sockets: *socket*, *bind*, *listen* y *accept*.

El servidor permanece en un bucle infinito esperando conexiones de clientes. Cada vez que se recibe una nueva conexión mediante la llamada a *accept*, se obtiene un descriptor de socket asociado a ese cliente. Este descriptor se utiliza posteriormente para recibir los datos enviados por el cliente y enviar la respuesta correspondiente.

2.2. Concurrencia y sincronización

El servidor está diseñado para atender múltiples clientes de forma concurrente gracias a la creación de un hilo independiente para cada conexión que entra: cuando el servidor acepta una nueva conexión, crea un nuevo hilo utilizando *pthread_create*. Este hilo se encarga de procesar desde la recepción del mensaje hasta el envío de la respuesta y el cierre de la conexión. Esto nos permite que varios clientes realicen operaciones de forma simultánea sin bloquear el funcionamiento del servidor, lo que hace que sea más escalable.

Dado que todos los hilos comparten el mismo almacenamiento de claves, es necesario garantizar la consistencia de los datos frente a accesos concurrentes. Para ello, *claves.c* usa mecanismos de sincronización basados en mutex (*pthread_mutex_t*), siguiendo lo realizado en la práctica anterior.

2.3. Gestión del almacenamiento de claves

El almacenamiento de las claves y sus valores asociados se lleva a cabo con la biblioteca *libclaves.so*, reutilizada de la práctica anterior. Cada clave puede estar asociada a: una cadena de texto (*value1*), un vector de números reales (*V_value2*), una estructura compuesta por tres enteros (*value3*). De esta manera, separamos la lógica del servidor de la del almacenamiento de claves.

2.4. Gestión de errores y finalización del servidor

Tras cada llamada a funciones críticas del sistema (*socket*, *bind*, *listen*, *accept*, *recv* y *send*), se comprueba el valor de retorno para detectar posibles fallos. En caso de error, el servidor informa del problema mediante mensajes en la salida estándar y evita continuar con una operación inválida. Las operaciones del servicio devuelven:

- **0** → operación realizada correctamente.
- **-1** → error lógico o clave inexistente.
- **1** → confirmación de existencia de una clave.
- **-2** → error de comunicación.

Además, al igual que se hizo en la anterior práctica, el servidor implementa *SIGINT*, que permite terminar el proceso de forma controlada cuando se presiona Ctrl+C. Este mecanismo asegura que los recursos del sistema se liberen correctamente y evita dejar sockets abiertos o procesos en estado inconsistente.

3. Diseño del proxy (*proxy-sock.c*)

3.1. Uso de variables de entorno

El proxy obtiene la dirección IP y el puerto del servidor a partir de variables de entorno definidas en el sistema: `IP_TUPLAS` y `PORT_TUPLAS`. De esta forma, el mismo cliente puede conectarse a diferentes servidores simplemente cambiando el valor de las variables de entorno.

3.2. Conexión con el servidor

Cada vez que el cliente invoca una operación de la API del servicio, el proxy establece una nueva conexión TCP con el servidor. Para ello, el proxy hace los siguientes pasos:

1. Obtiene la dirección IP y el puerto del servidor a partir de las variables de entorno.
2. Crea un socket TCP con la función `socket`.
3. Establece la conexión con el servidor con la función `connect`.
4. Envía la petición a la operación solicitada.
5. Recibe la respuesta enviada por el servidor.
6. Cierra la conexión con la función `close`.

3.3. Conversión de llamadas de la API a mensajes

Cuando el cliente invoca una función de la API, el proxy realiza las siguientes tareas: convierte los parámetros de la función en una representación textual, construye el mensaje correspondiente al protocolo de aplicación, envía el mensaje al servidor mediante el socket, recibe la respuesta del servidor, interpreta la respuesta y devuelve el resultado al cliente

Por ejemplo, si el cliente invocara la función: `modify_value(key, value1, N, V_value2, value3)`, el proxy genera un mensaje que contiene: el código de operación correspondiente a la función, la longitud de la clave, la clave, la longitud de `value1`, la cadena `value1`, el número de elementos del vector `V_value2`, los elementos de `V_value2` y los valores de la estructura `value3` → Este mensaje se enviaría al servidor a través del socket, se interpretaría el código de operación y se ejecutaría la función.

4. Protocolo de aplicación

Para garantizar que el protocolo de comunicación sea totalmente independiente del lenguaje de programación, se ha diseñado una serialización basada en texto plano. En ningún caso se envían volcados de memoria o estructuras propias de C (`struct`) a través del socket.

4.1. Formato de mensajes

Todos los datos intercambiados entre el cliente y el servidor se codifican como caracteres ASCII utilizando el salto de línea (`\n`) como delimitador genérico:

- Tipos básicos numéricos (`int`, `float`): se convierten a su representación en cadena de texto (ej. usando `snprintf`) y se transmiten seguidos de un `\n`.
- Cadenas de texto (strings): para evitar ambigüedades con los terminadores nulos (`\0`) y caracteres especiales, se emplea un formato de dos líneas. Primero se envía un entero con la longitud exacta de la cadena útil y, en la siguiente línea, los caracteres del string seguidos de un `\n`.

- Vectores numéricos (*V_value2*): se transmiten iterando secuencialmente sobre cada elemento del array. Cada *float* se envía en una línea individual.
- Estructura *Paquete* (*value3*): se desensambla en origen. Sus tres atributos enteros (*x*, *y*, *z*) se serializan y envían como variables escalares separadas por *\n*. El receptor reconstruye la entidad agrupando los tres valores recibidos.

4.2. Operaciones soportadas

El flujo de comunicación siempre es iniciado por el cliente enviando un Código de Operación (OP) entero, seguido de los argumentos requeridos por la función solicitada.

API a invocar	Código OP	Flujo de datos enviados por el socket (separados por <i>\n</i>)
<i>destroy()</i>	1	1
<i>set_value()</i>	2	2 → <i>len(key)</i> → <i>key</i> → <i>len(v1)</i> → <i>v1</i> → <i>N_v2</i> → <i>V_v2[0]</i> ... <i>V_v2[N-1]</i> → <i>x</i> → <i>y</i> → <i>z</i>
<i>get_value()</i>	3	3 → <i>len(key)</i> → <i>key</i>
<i>modify_value()</i>	4	4 → <i>len(key)</i> → <i>key</i> → <i>len(v1)</i> → <i>v1</i> → <i>N_v2</i> → <i>V_v2[0]</i> ... <i>V_v2[N-1]</i> → <i>x</i> → <i>y</i> → <i>z</i>
<i>delete_key()</i>	5	5 → <i>len(key)</i> → <i>key</i>
<i>exist()</i>	6	6 → <i>len(key)</i> → <i>key</i>

4.3. Respuestas del servidor

El servidor procesa la operación solicitada y emite una respuesta al cliente. El primer elemento de cualquier respuesta es siempre un código de resultado entero (0 para indicar éxito, -1 para error general o clave inexistente, y 1 para confirmación de existencia).

Tipo de operación	Formato de la respuesta del servidor
Generales (<i>set</i> , <i>modify</i> , <i>delete</i> , <i>destroy</i> , <i>exist</i>)	Únicamente el código de resultado entero (0, -1 o 1).
Lectura con error (<i>get_value</i>)	Únicamente envía -1 (indicando fallo o clave inexistente).
Lectura exitosa (<i>get_value</i>)	Tras enviar el código 0, el servidor envía los datos asociados a la clave: <i>len(key)</i> → <i>v1</i> → <i>N_v2</i> → <i>V_v2[0]</i> ... <i>V_v2[N-1]</i> → <i>x</i> → <i>y</i> → <i>z</i>

5. Compilación y ejecución

5.1. Uso del Makefile

En caso de que haya que eliminar los ficheros generados durante la compilación: `make clean`. Para compilar el proyecto completo, ejecutamos desde el directorio raíz del proyecto: `make`.

El Makefile de esta práctica ha sido simplificado respecto al de la práctica anterior, ya que lo hemos adaptado al uso de sockets TCP en lugar de colas de mensajes. Se han eliminado las dependencias específicas de las colas de mensajes y se han mantenido únicamente las bibliotecas necesarias para la comunicación mediante sockets y la gestión de hilos en el servidor.

5.2. Cómo lanzar servidor y cliente

El servidor se ejecuta indicando como argumento el puerto TCP en el que debe escuchar conexiones entrantes. Por ejemplo: `./servidor 4500`.

Cuando el servidor está escuchando, el cliente debe configurar las variables de entorno que indican la dirección IP y el puerto del servidor: `export IP_TUPLAS=127.0.0.1 export PORT_TUPLAS=4500`. A continuación, se ejecuta el cliente mediante: `./cliente`. El sistema permite ejecutar múltiples clientes de forma concurrente. Por ejemplo, para lanzar cinco clientes simultáneamente:

```
for i in {1..5}; do env IP_TUPLAS=127.0.0.1 PORT_TUPLAS=4500 ./cliente & done wait
```

6. Trabajo extra

Para realizar la evaluación de rendimiento de la implementación actual basada en sockets TCP, se reutilizaron los resultados obtenidos en la práctica anterior, donde el sistema utilizaba colas de mensajes POSIX. De esta forma, se pudo comparar el comportamiento de ambas tecnologías de comunicación bajo condiciones similares.

Las pruebas se realizaron midiendo el tiempo total de ejecución de la batería de pruebas del cliente utilizando el comando `time bash -c 'for i in {1..x}; do env IP_TUPLAS=127.0.0.1 PORT_TUPLAS=4500 ./cliente & done; wait'`, donde x es el número de clientes que se desea probar. Y se repitieron los experimentos con diferentes números de clientes concurrentes para analizar la escalabilidad del servidor. Obtuvimos los siguientes valores:

Configuración	Tiempo
cliente_local (P1)	real 0m0,005s
cliente_mq (P1)	real 0m0,011s
20 cliente_mq (P1)	real 0m0,073s
cliente_tcp (P2)	real 0m0,014s
5 cliente_tcp (P2)	real 0m0,019s
20 cliente_tcp (P2)	real 0m0,051s
100 cliente_tcp (P2)	real 0m1,136s

Los resultados obtenidos muestran que la implementación basada en sockets TCP presenta un tiempo de ejecución ligeramente superior al de la versión local, debido al coste asociado a la comunicación por red. No obstante, los tiempos obtenidos son inferiores a los de la versión basada en colas de mensajes, lo que indica una mejora en el rendimiento del sistema.

Además, al aumentar el número de clientes concurrentes, el sistema mantiene un comportamiento estable y completa correctamente la ejecución de todas las peticiones.

*Nota:

- Se ha hecho uso de la IA (Github Copilot, ChatGPT) como herramienta de apoyo para la realización de la práctica, sin que en ningún momento ponga en duda nuestra autoría de esta (resolver dudas específicas, ayudar a la redacción de la memoria).